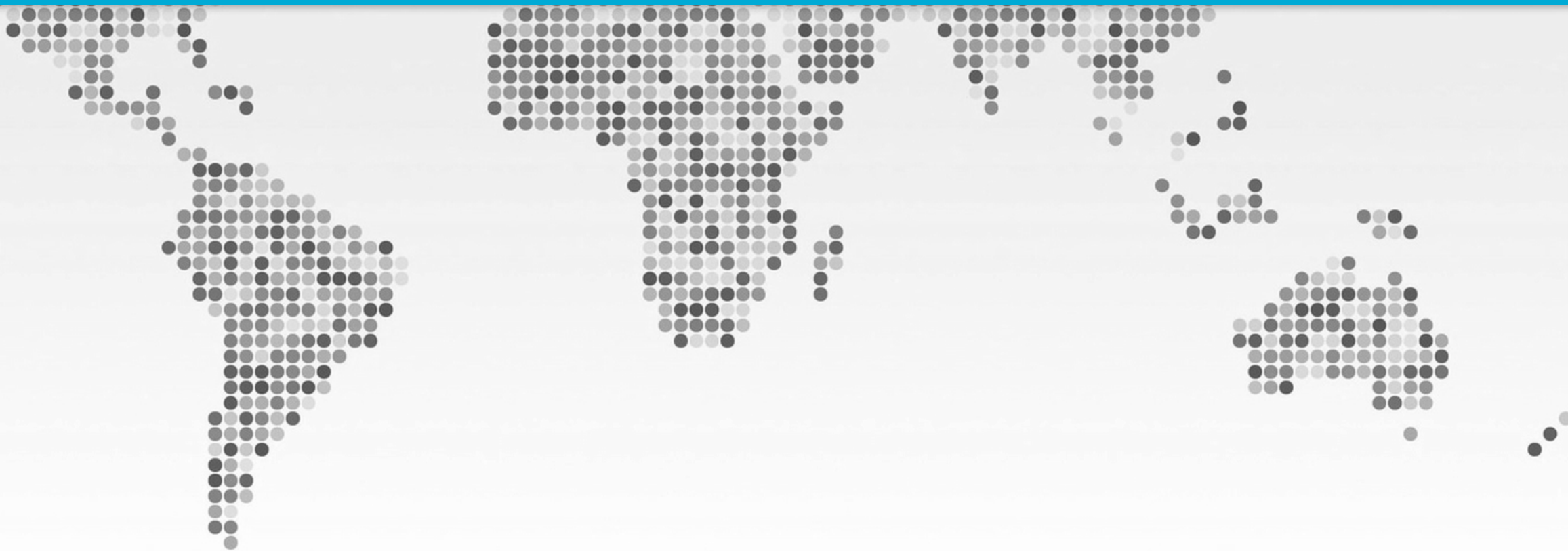


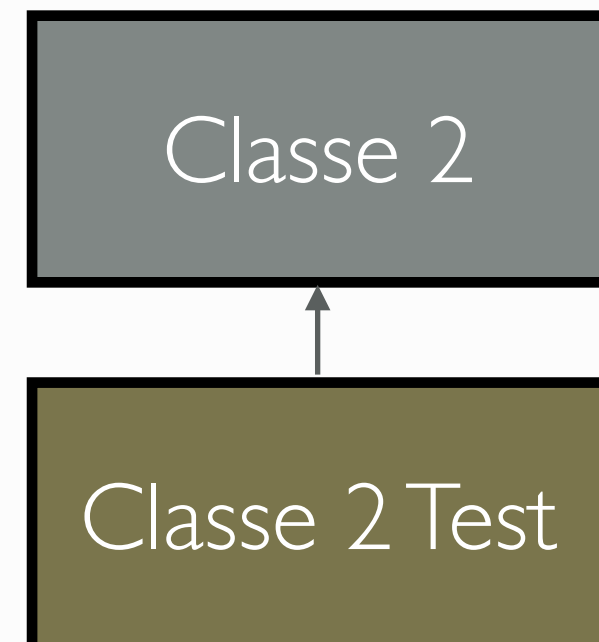
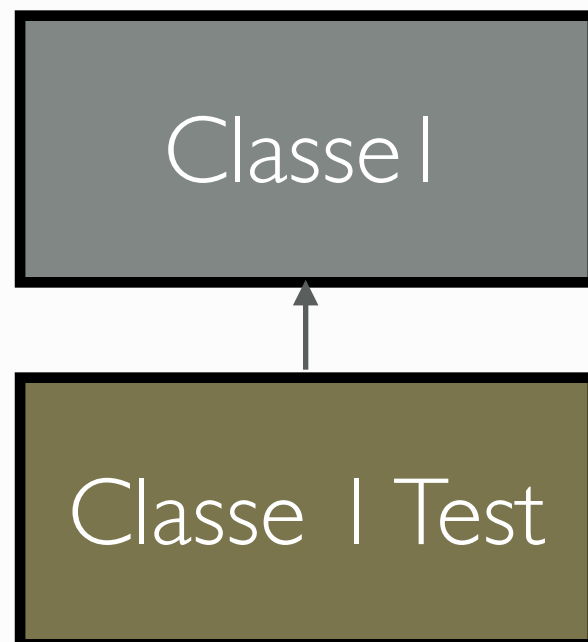
# Mockito

## Ingegneria del Software



Finora abbiamo visto degli esempi dove la componente da testare non aveva nessuna dipendenza con altre componenti, ma in linea generale potrebbe non essere sempre vero.

Finora abbiamo visto degli esempi dove la componente da testare non aveva nessuna dipendenza con altre componenti, ma in linea generale potrebbe non essere sempre vero.

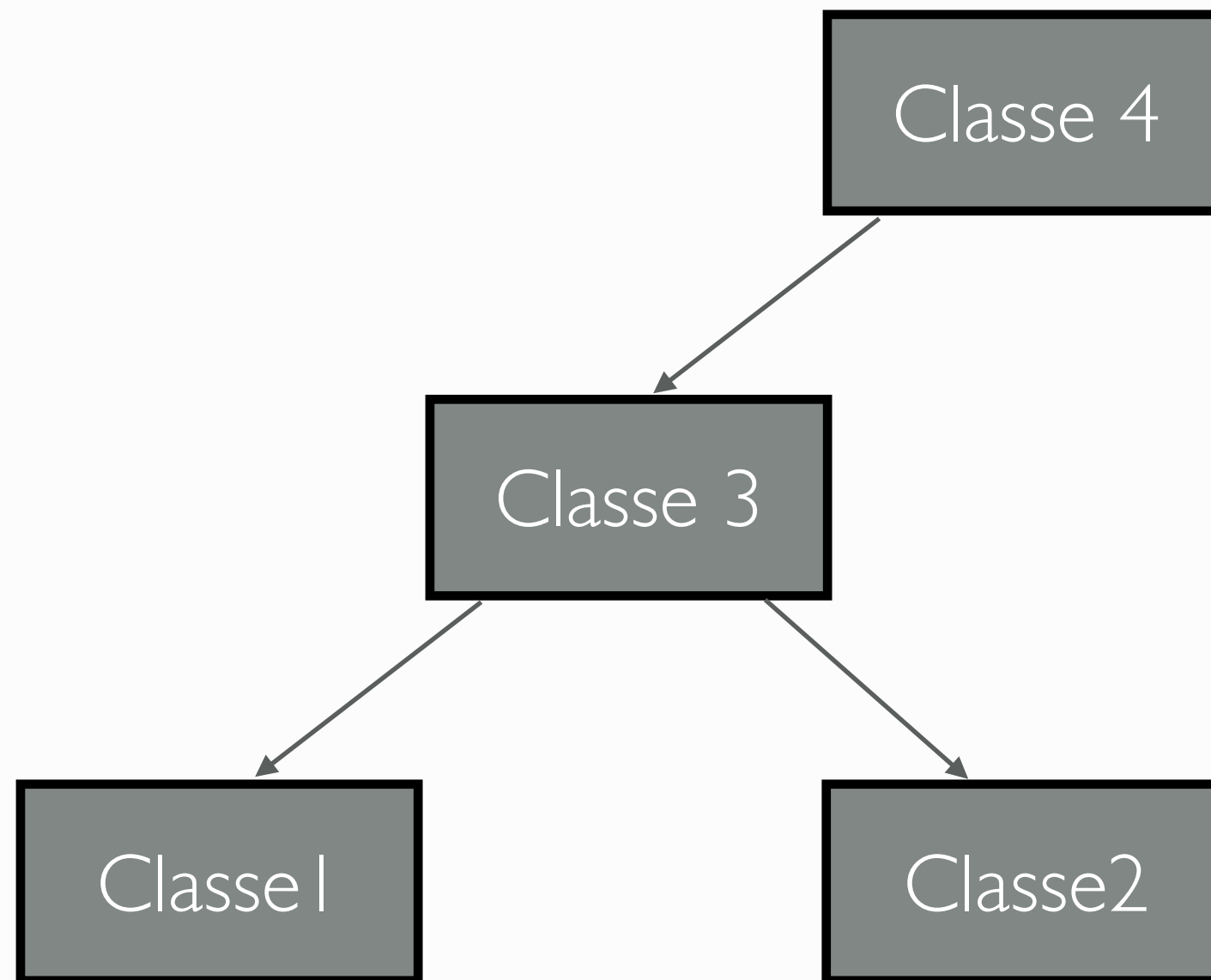


# Mock Object

Finora abbiamo visto degli esempi dove la componente da testare non aveva nessuna dipendenza con altre componenti, ma in linea generale potrebbe non essere sempre vero.

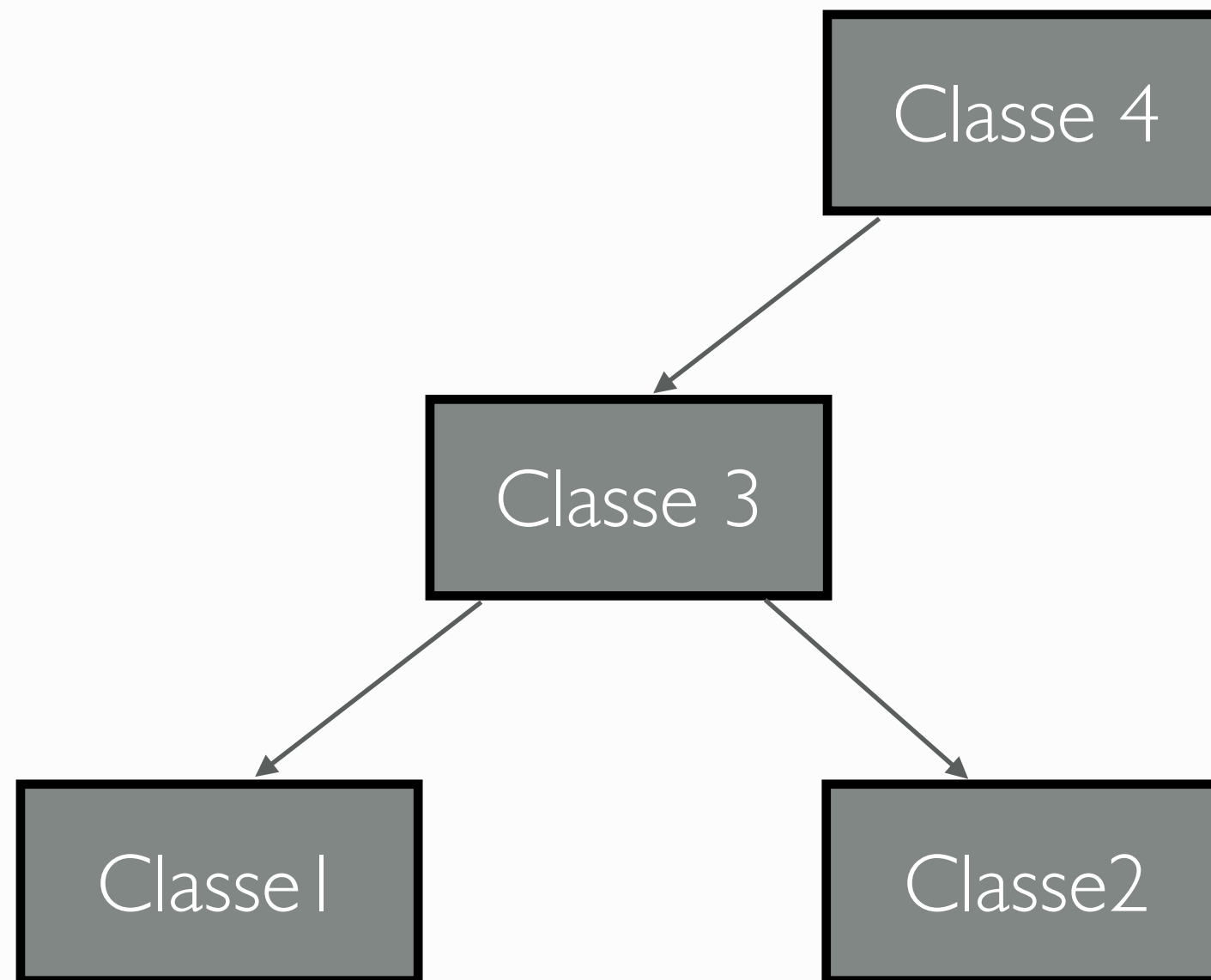


Il software spesso è complesso ed ha bisogno di molte componenti per poter funzionare correttamente.



La **Classe 4** utilizza la **Classe 3**.

La **Classe 3** a sua volta utilizza **Classe 1** e **Classe 2**.

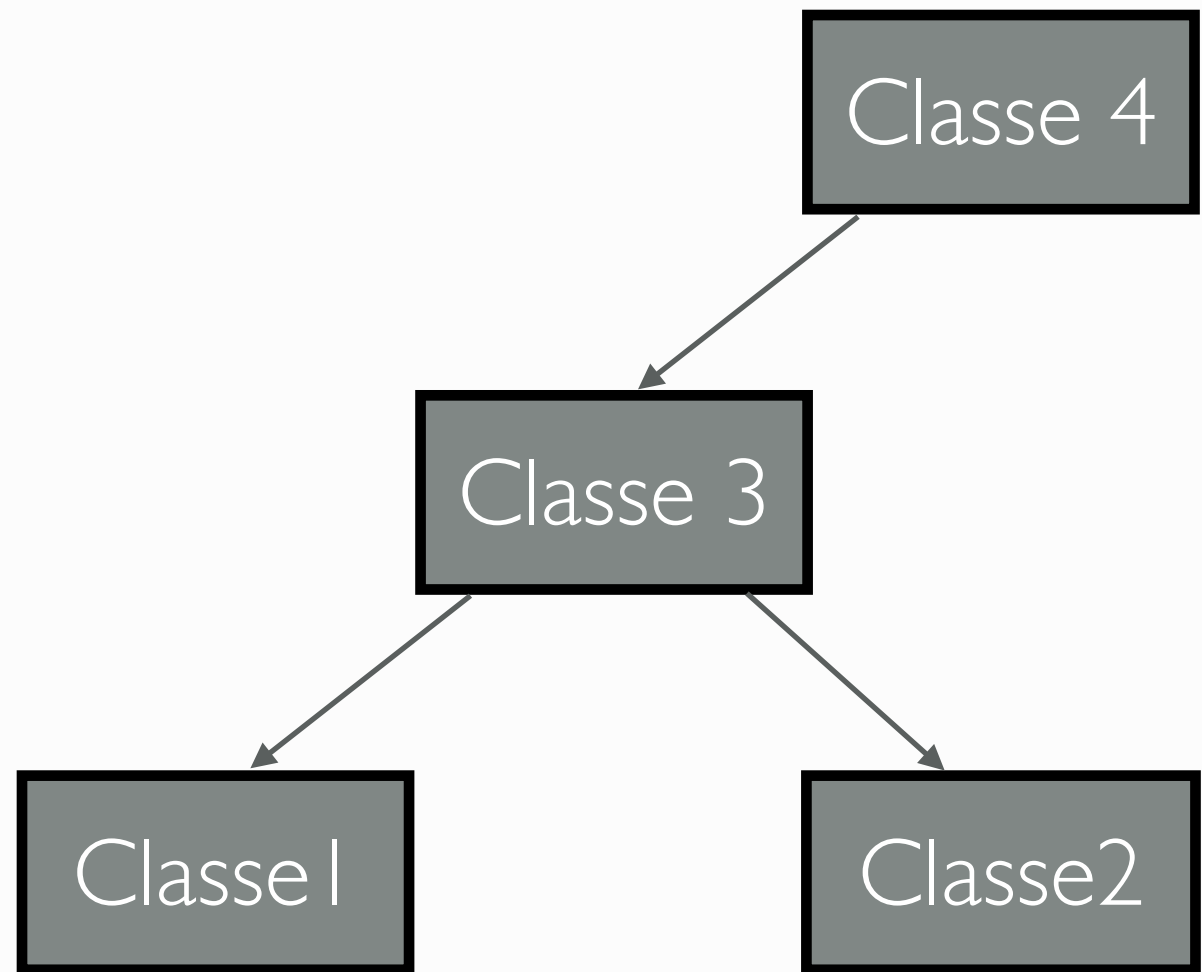


La **Classe 4** utilizza la **Classe 3**.

La **Classe 3** a sua volta utilizza **Classe 1** e **Classe 2**.

Ci troviamo in una situazione complessa, poichè lo scopo dello Unit Test è testare le singole componenti in **isolamento**.

Come faccio a testare la **Classe 3** isolandola dalle altre due?



La **Classe 4** utilizza la **Classe 3**.

La **Classe 3** a sua volta utilizza **Classe 1** e **Classe 2**.

Ci troviamo in una situazione complessa, poichè lo scopo dello Unit Test è testare le singole componenti in **isolamento**.



La **Classe 4** utilizza la **Classe 3**.

La **Classe 3** a sua volta utilizza **Classe 1** e **Classe 2**.

Ci troviamo in una situazione complessa, poichè lo scopo dello Unit Test è testare le singole componenti in **isolamento**.

## Perché creare un mock?

Un Mock è un'istanza di una classe il cui comportamento di alcuni dei suoi metodi viene definito direttamente nel test.

Creare un oggetto Mock può essere utile in questi casi:

- Lavoro in parallelo
- Non si vogliono fare modifiche al production code per far ritornare un determinato valore ad una certa dipendenza
- Scrivere ed eseguire in maniera “rapida” test unitari
- Miglioramento del design del codice sorgente
- Simulare accessi ad un database
- Etc...

## Differenze tra Mock e Stub?

### Mock

- Comportamento dinamico
- Può utilizzare il Polimorfismo
- Può implementare tutti i metodi

### Stub

- Comportamento predeterminato
- Sostituisce il codice sorgente
- Solo i metodi usati dai test vengono implementati

## Differenze tra Mock e Stub?

### Mock

- Comportamento dinamico
- Può utilizzare il Polimorfismo
- Può implementare tutti i metodi

### Stub

- Comportamento predeterminato
- Sostituisce il codice sorgente
- Solo i metodi usati dai test vengono implementati

```
public class InvoiceDAOStub {  
  
    public List<Invoice> all() {  
  
        Invoice i1 = new Invoice("M", 20.0);  
        Invoice i2 = new Invoice("A", 300.0);  
  
        return Arrays.asList(i1, i2);  
    }  
}
```

<https://martinfowler.com/articles/mocksArentStubs.html>

mockito



## Mockito...Come funziona?

Mockito funziona con il principio di GWT, ovvero:

- **Given:** Descrive lo stato o l'ambiente PRIMA dell'inizio del mock (una sorta di pre-condizione)
- **When:** Descrive il trigger che deve avvenire al fine di scatenare una risposta
- **Then:** Descrive i cambiamenti che devono essere effettuati

Tra i vari metodi che Mockito mette a disposizione, i fondamentali sono i seguenti:

- `mock(ClassToMock)`
- `when(Method call)`
- `thenReturn(value)`

## Mockito...Come funziona?

Mockito funziona con il principio di GWT, ovvero:

- **Given:** Descrive lo stato o l'ambiente PRIMA dell'inizio del mock (una sorta di pre-condizione)
- **When:** Descrive il trigger che deve avvenire al fine di scatenare una risposta
- **Then:** Descrive i cambiamenti che devono essere effettuati

Tra i vari metodi che Mockito mette a disposizione, i fondamentali sono i seguenti:

- `mock(ClassToMock)`
- `when(Method call)`
- `thenReturn(value)`

## Come si configura?

### POM.XML

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>4.9.0</version>
  <scope>test</scope>
</dependency>
```

## Facciamo un esempio

Consideriamo il seguente caso di test. Le operazioni di Given, When e Then sono le seguenti:

```
@test
public void testCalcolatrice(){
    //Given
    Calcolatrice calc = new Calcolatrice();
    SecondaDipendenza sec = new SecondaDipendenza()
    //When
    int a= calc.somma(11,1);

    //then
    assertEquals(12,a);
}
```

## Facciamo un esempio

Possiamo eseguire il seguente mock

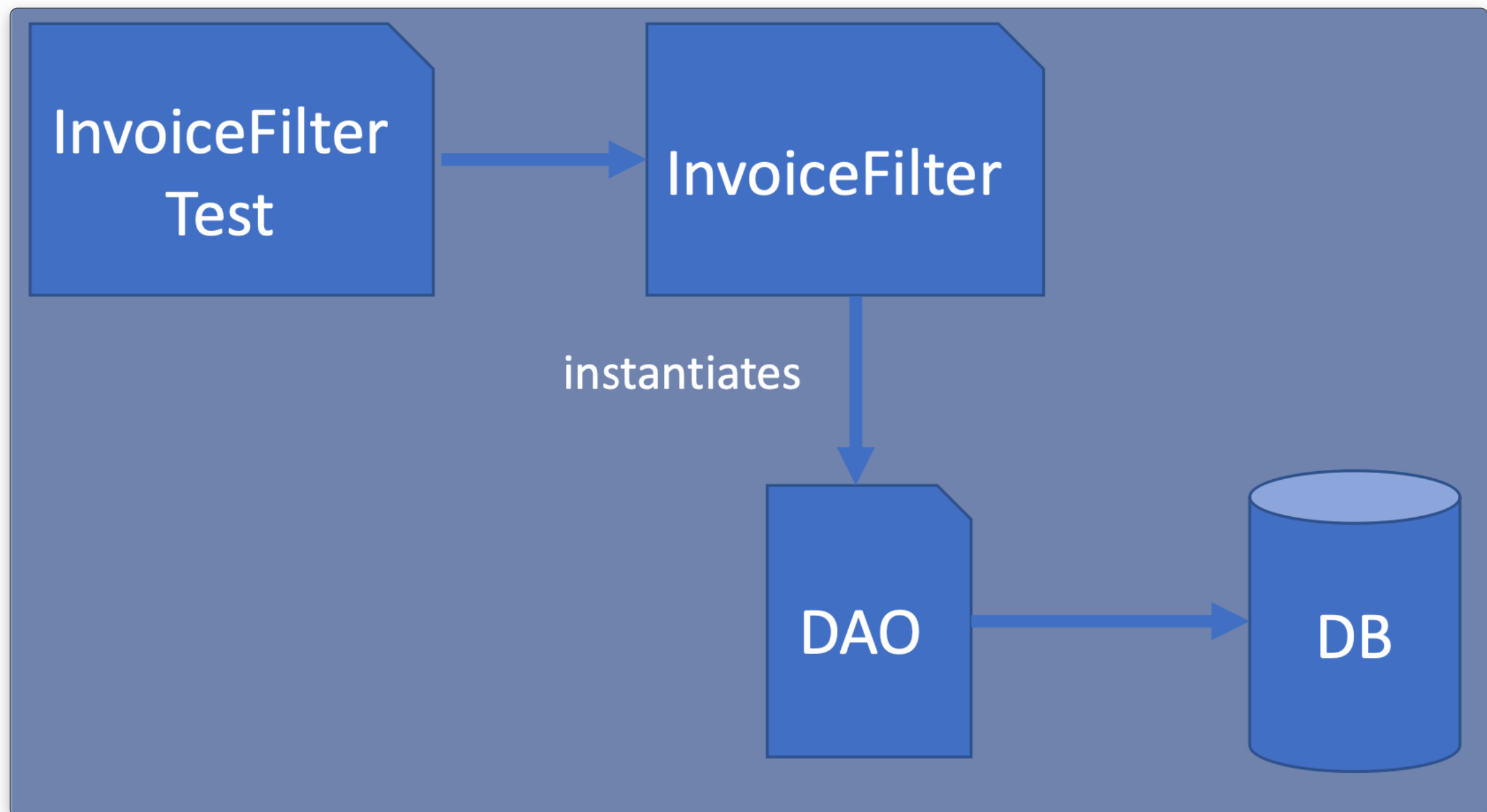
```
@test
public void testCalcolatrice(){
    //Given
    Calcolatrice calc = new Calcolatrice();
    SecondaDipendenza mockDep = new Mock(SecondaDipendenza.class);
    when(mockDep.method()).thenReturn("ok");
    //When
    int a= calc.somma(11,1);

    //then
    assertEquals(12,a);
}
```

In questo caso abbiamo simulato la chiamata di un metodo di “SecondaDipendenza”, e abbiamo simulato che questo metodo ci abbia restituito un “ok”.

## Facciamo un esempio

Immaginiamo di dover creare un classe di Test per la classe InvoiceFilter che contiene il metodo *filter()*.



Facciamo un esempio

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

## Un altro esempio

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```



## Un altro esempio

```
public List<Invoice> all() {  
    InvoiceDao dao = Mockito.mock(InvoiceDao.class);  
    List<Invoice> all = dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for (Invoice inv : all) {  
        if (inv.getValue() < 100.0) {  
            filtered.add(inv);  
        }  
    }  
  
    return filtered;  
}
```

Si potrebbe pensare che  
sono questi le due parti più  
complesse da testare



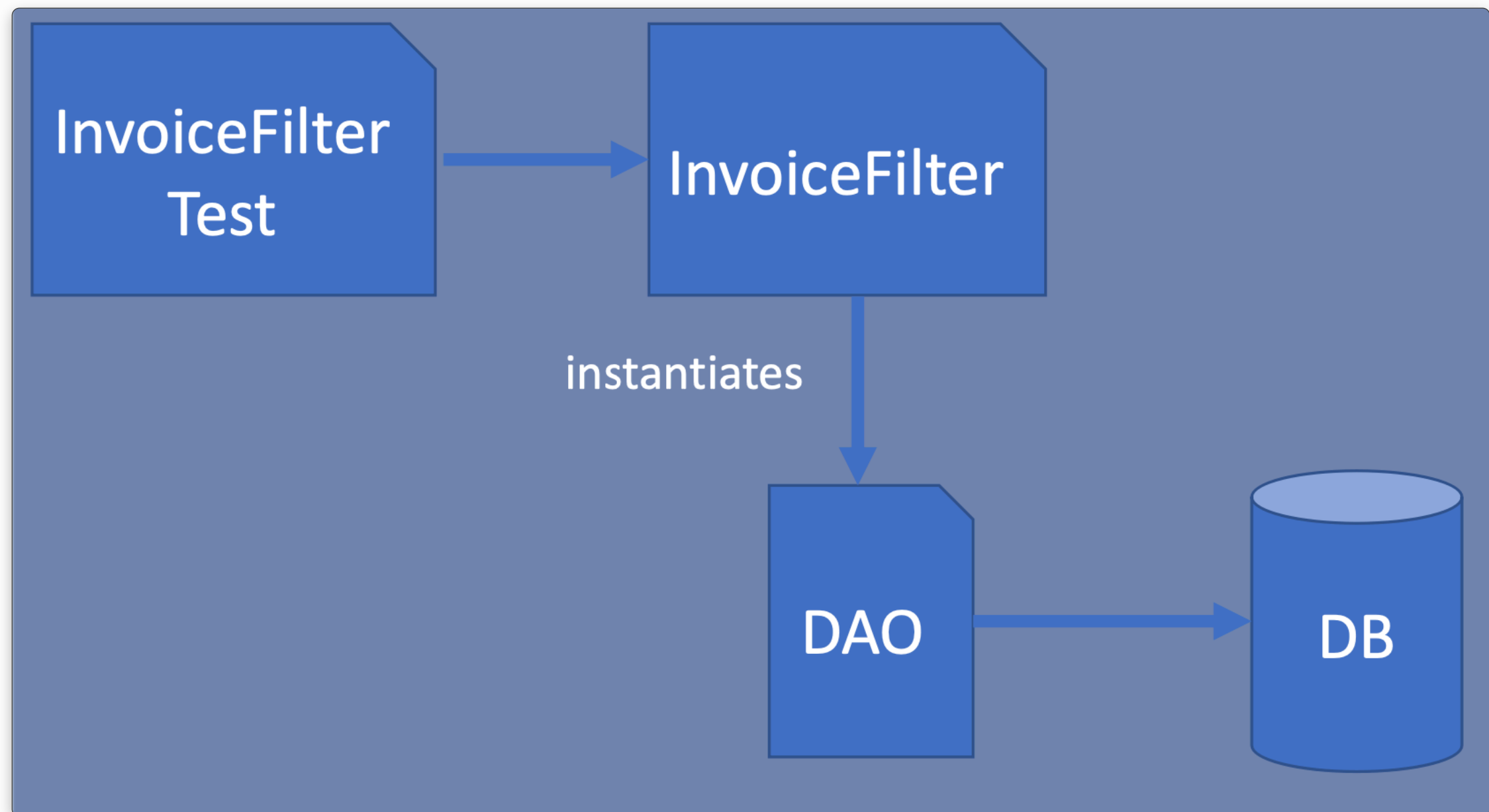
## Un altro esempio

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if (inv.getAmount() > 100.0) {  
            filtered.add(inv);  
        }  
    }  
  
    return filtered;  
}
```

E invece è questa la parte più complessa!

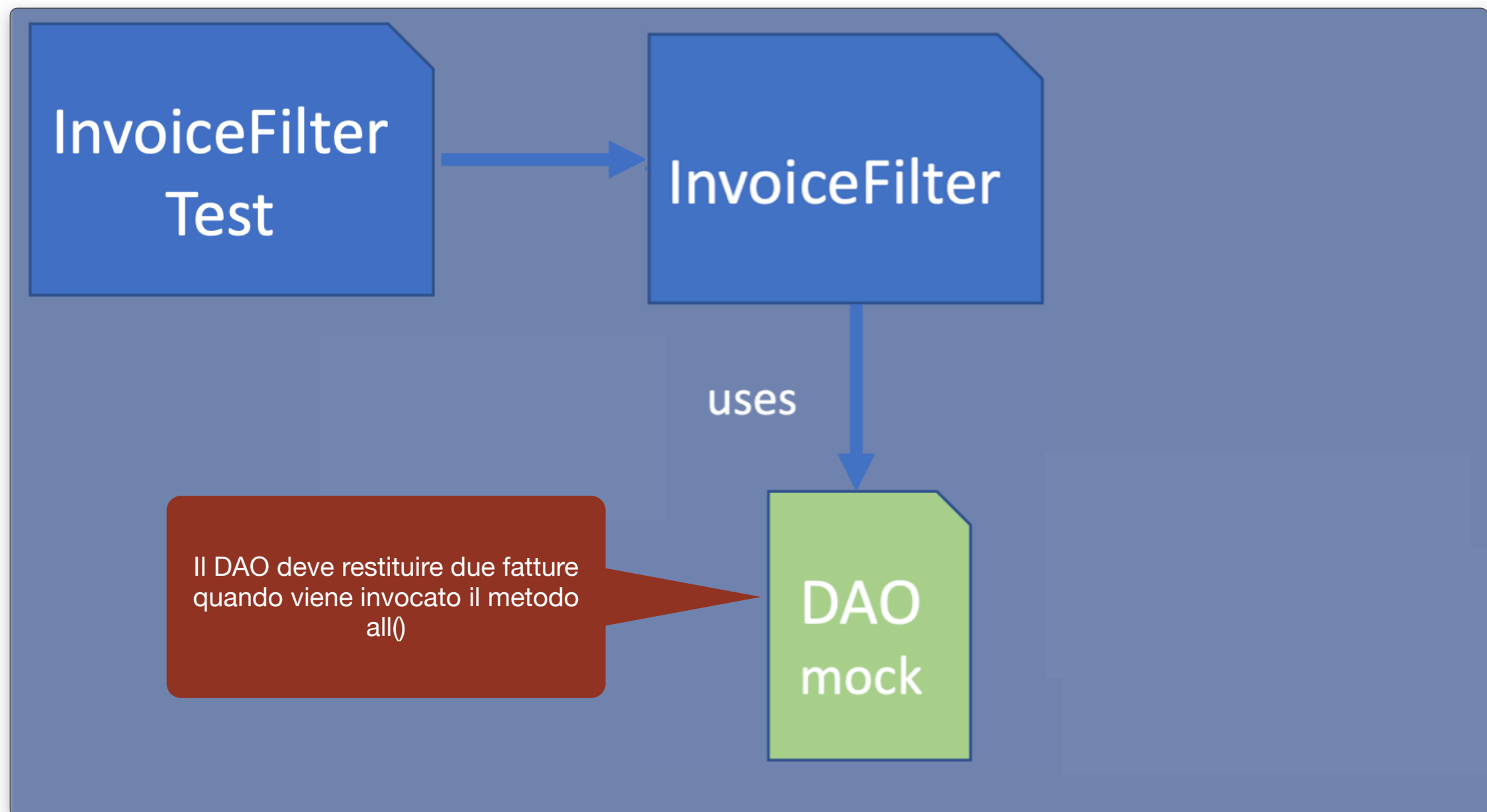
## Facciamo un esempio

La complessità deriva dalla dipendenza del metodo InvoiceFilter sulla classe DAO per poter interagire con il DB.



## Facciamo un esempio

Creiamo un oggetto mock che simulerà l'accesso al database. Questa simulazione farà solo cose semplici e finte, come restituire un elenco di fatture che sono in memoria.



Facciamo un esempio

```
@Test void filterInvoices() {  
    InvoiceDao dao = new InvoiceDao();  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    dao.save(i1); dao.save(i2);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
  
    assertEquals(1, result.size());  
    assertEquals(i1, result.get(0));  
    dao.close();  
}
```

Facciamo un esempio

```
@Test void filterInvoices() {  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    InvoiceDao dao =  
        Mockito.mock(InvoiceDao.class);  
  
    List<Invoice> list = Arrays.asList(i1,i2);  
    Mockito.when(dao.all()).thenReturn(list);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
    // assertions  
}
```

## Facciamo un esempio

```
@Test void filterInvoices() {  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    InvoiceDao dao =  
        Mockito.mock(InvoiceDao.class);  
  
    List<Invoice> list = Arrays.asList(i1,i2);  
    Mockito.when(dao.all()).thenReturn(list);  
  
    InvoiceFilter f = new InvoiceFilter();  
    List<Invoice> result = f.filter();  
    // assertions  
}
```

Abbiamo un altro problema: se ricordate, il metodo filter utilizza anche lui il DAO.

Facciamo un esempio

```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all = dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for (Invoice inv : all) {  
        if (inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

Facciamo un esempio

```
public List<Invoice> filter() {  
  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

Facciamo un esempio

```
public class InvoiceFilter {  
    private InvoiceDao dao;  
  
    public InvoiceFilter  
        (InvoiceDao dao) {  
        this.dao = dao;  
    }  
  
    public List<Invoice> filter() { ... }  
}
```

Al posto di usarlo nel metodo filter, lo usiamo direttamente nel costruttore della classe InvoiceFilter così che ogni metodo della classe usi questa implementazione.

## Facciamo un esempio

```
@Test void filterInvoices() {  
    Invoice i1 = new Invoice("M", 20.0);  
    Invoice i2 = new Invoice("A", 300.0);  
    InvoiceDao dao =  
        Mockito.mock(InvoiceDao.class);  
  
    List<Invoice> list = Arrays.asList(i1,i2);  
    Mockito.when(dao.all()).thenReturn(list);  
  
    InvoiceFilter f = new InvoiceFilter(dao);  
    List<Invoice> result = f.filter(),  
    // assertions  
}
```

Nel test case passiamo l'oggetto mockato ad InvoiceFilter che a sua volta chiamerà il metodo filter per ottenere una lista di Invoice

Nel caso di test questo metodo viene eseguito tramite il Mock!

Facciamo un esempio

```
public List<Invoice> filter() {  
  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

Grazie al polimorfismo, quando passiamo il dao dal caso di test, in realtà usiamo il dao mockato e non la vera implementazione.

# Mockito

## Ingegneria del Software

